BOSTON INSTITUTE OF ANALYTICS

Capstone Project Report

WebSentinel

An Automated Website Vulnerability Scanner

A Lightweight Python CLI Tool for Web Application Security Assessment

---

Project Topic	Create a tool that can be used to perform a website vulnerability scan
Project Name	WebSentinel
Name	Mohamed Shaheeth M
Course	Cyber Security & Ethical Hacking
Institute	Boston Institute of Analytics
Platform	Kali Linux (Debian-based)
Primary Language	Python 3.11+
Demonstration Target	http://scanme.nmap.org (authorised test host)
Final Output	PDF/HTML/JSON security assessment report + terminal findings
Date	_____

Table of Contents

1. Abstract 3
2. Introduction 3
3. Research — Target Website 4
4. Data Collection — Technology Stack 5
5. WebSentinel Design & Architecture 6
6. Impact Analysis 7
7. Proof of Concept (PoC) 8
8. Recommendations — Security Measures & Mitigation 16
9. Code & Tools Used 16
10. Comparison with Existing Kali Tools 19
11. Limitations, Testing & Validation 20
12. Conclusion 21
13. References 22
Appendix A — Full Scan Report Output 23
Appendix B — Environment & Reproduction 25

1. Abstract

WebSentinel is a lightweight, Python-based website vulnerability assessment tool developed for authorised security testing and educational use on Kali Linux. It addresses the capstone requirement to build a simple tool capable of performing a website vulnerability scan, while keeping the assessment non-destructive, rate-limited and modular.

The tool combines target validation with SSRF protection, DNS resolution, conservative Nmap reconnaissance, HTTP/HTTPS analysis, security-header checks, cookie-security analysis, TLS/certificate inspection, passive technology detection, selected configuration checks, OWASP Top 10 mapping and a custom WebSentinel Risk Score. Scanners collect raw evidence; a separate checks layer converts that evidence into standardised findings carrying severity, confidence, OWASP mapping, evidence, impact and remediation.

For the demonstrated assessment, WebSentinel performed a full scan of http://scanme.nmap.org — a host the Nmap Project explicitly authorises for scan testing. The tool resolved the target to 45.33.32.156, ran for 131.1 seconds, and produced a security score of 72/100 at a MODERATE risk level, with seven findings (two Medium, five Low) all mapped to OWASP A05:2021 – Security Misconfiguration.

The final deliverable includes a terminal interface, structured JSON/HTML/PDF reporting, SQLite-backed scan history and a packaging path that installs the tool globally on Kali Linux with an application-menu launcher. The project deliberately excludes exploitation, credential attacks, authentication bypass and destructive testing, positioning it as a first-stage assessment and learning tool rather than a replacement for a full penetration-testing platform.

Keywords: *Web Security*, *Vulnerability Assessment*, *Python*, *Kali Linux*, *OWASP Top 10*, *Nmap*, *Security Headers*, *TLS*, *Passive Technology Detection*, *Risk Scoring*.

2. Introduction

2.1 Background

Modern web applications expose several layers that require assessment: network services, HTTP behaviour, browser-facing security controls, cookies, encryption, server configuration and technology disclosure. A practical first-stage scanner should therefore collect multiple categories of evidence without immediately moving into intrusive exploitation. WebSentinel was designed around this first-stage assessment model.

2.2 Problem Statement

Many learners can run individual security tools but do not see how reconnaissance data becomes a structured vulnerability finding. This project builds the complete

pipeline: validate a target, collect evidence, interpret configuration weaknesses, classify findings, calculate a risk score and generate a readable report — so every step is transparent and understandable.

2.3 Project Objectives

1. Develop a lightweight website vulnerability assessment utility in Python.
2. Provide a simple command-line workflow suitable for Kali Linux.
3. Perform non-destructive reconnaissance and configuration checks.
4. Detect missing or weak security headers and related web-configuration issues.
5. Inspect cookies and TLS/certificate properties without storing cookie values.
6. Perform passive technology detection with confidence labels.
7. Map findings to relevant OWASP Top 10 (2021) categories.
8. Generate structured JSON, HTML and PDF assessment reports and store scan history in SQLite.
9. Demonstrate the complete workflow on an explicitly authorised public test target.

2.4 Scope

- **Included:** target validation, DNS, Nmap service discovery, HTTP/HTTPS analysis, security headers, cookies, TLS/certificates, passive technology detection, selected disclosure/configuration checks and a curated full-scan exposed-path check.
- **Excluded:** authentication testing, authorization testing, business-logic testing, fuzzing, exploit execution, credential attacks and denial-of-service activity.
- **Authorisation:** the scanner is intended only for systems the tester owns or is explicitly authorised to assess.

3. Research — Target Website

To exercise and demonstrate the scanner, scanme.nmap.org was selected as the target. It is a host operated by the Nmap Project specifically so that anyone learning network-scanning tools can test them legally, which makes it a safe, authorised and community-standard demonstration target.

3.1 Target Profile

Attribute	Detail
Name / Hostname	scanme.nmap.org
URL used in PoC	http://scanme.nmap.org
Resolved IP	45.33.32.156
Category / Type	Public, authorised scan-practice host (not a production

Attribute	Detail
	business site)
Operated by	The Nmap Project (author: Gordon “Fyodor” Lyon)
Purpose	Provided so people may legally test port/vulnerability scanners against a real host
Hosting / OS	Linux (Ubuntu); web server Apache httpd 2.4.7; SSH via OpenSSH 6.6.1p1

3.2 Usage & Popularity

- **Canonical learning target —** scanme.nmap.org is referenced in the official Nmap documentation and is used in a large share of security tutorials, courses and labs worldwide as the first host students scan.
- **Explicitly authorised —** the host displays a note permitting scanning, on the condition that users do not scan it excessively (a few scans per day), reflecting responsible-use etiquette.
- **Real, not synthetic —** it is a genuine internet-facing Linux server, so results (open ports, server banner, missing headers) are real observations rather than a simulated response.

Because the target is real yet authorised, it is ideal for demonstrating an active scanner responsibly — the tester gets true findings without the legal risk of scanning a third-party site.

3.3 Why Authorisation Matters

Active scanning sends real packets to a target: a port scan opens connections, an HTTP request is logged by the server, and a TLS handshake is negotiated. Even though none of this is destructive, it is still visible to the target’s owner and can trip intrusion-detection or rate-limiting systems. In many jurisdictions, scanning a system without permission can breach computer-misuse law regardless of intent. Choosing scanme.nmap.org — a host whose owner explicitly invites scanning — means the entire demonstration stays lawful and ethical, and models exactly the discipline a professional would apply before running any scan.

4. Data Collection — Technology Stack

Fingerprinting the target’s technology stack is a key stage of assessment, because the relevant weaknesses and fixes depend on the software in use. This section covers both the stack WebSentinel identified on the target and the method by which it collects that data.

4.1 Technology Stack Identified on the Target

Layer	Detected value (source)
Web server	Apache httpd 2.4.7 (from the HTTP Server header)
Operating system	Ubuntu Linux (Server header: “Apache/2.4.7 (Ubuntu)”)
SSH service	OpenSSH 6.6.1p1 on 22/tcp (from earlier Nmap service scans)
Open ports	22/tcp (ssh) and 80/tcp (http)
Technology confidence	Apache — High (WebSentinel passive technology detection)

Note: in the final full-mode run the Nmap module exceeded its 120-second timeout, so port/service data above is drawn from WebSentinel’s standard-mode runs against the same host; the Apache/Ubuntu identification comes directly from the live HTTP response captured in the final scan (finding WS-007).

4.2 How WebSentinel Collects Target Data

Data collected	How WebSentinel obtains it
DNS / IP resolution	Resolves the hostname to its IP before scanning (also used for SSRF safety checks).
Open ports & services	Shells out to Nmap (argument array, no shell) for a bounded TCP service scan.
HTTP behaviour	Sends a standard HTTP/HTTPS GET; records status, redirects and HTTP→HTTPS enforcement.
Security headers	Passively reads response headers (CSP, HSTS, X-Frame-Options, etc.).
Cookies	Inspects Set-Cookie attributes — Secure, HttpOnly, SameSite (never stores cookie values).
TLS / certificate	Initiates a TLS handshake to read certificate validity, hostname match and protocol version.
Technology fingerprint	Passively infers server/framework from headers and signatures, with a confidence label.
Exposed paths	In full mode, probes a small curated set of sensitive paths with a soft-404 baseline.

Only the Nmap, HTTP and TLS stages actively contact the target; header, cookie and technology detection are passive analysis of data already received. WebSentinel records only security-relevant metadata, never sensitive values such as cookie contents.

5. WebSentinel Design & Architecture

WebSentinel is organised as a pipeline: a safety gate validates the target, scanner modules collect raw data, a checks layer converts that data into standardised

findings with OWASP mapping, a scoring engine assigns a security score, and a reporting layer renders the output and stores it in SQLite.

5.1 Architecture Pipeline

Target URL → **Target Validation (SSRF guard)** → **Scanner Modules** → **Checks Engine (OWASP mapping)** → **Risk Scoring** → **Reports (Terminal / JSON / HTML / PDF) + SQLite history**

5.2 Scan Modules

Every finding produced by the checks engine is tagged with the relevant OWASP Top 10 (2021) category. The core modules are:

#	Module	What it does	Primary OWASP mapping
1	Target validation / SSRF guard	Blocks private, internal and cloud-metadata IPs by default	Safety control
2	Nmap port/service scan	Enumerates open ports and service versions	A05: Security Misconfiguration
3	HTTP/HTTPS analysis	Status codes, redirects, HTTP→HTTPS enforcement	A05 / A02
4	Security header checks	CSP, HSTS, X-Frame-Options, X-Content-Type-Options, etc.	A05: Security Misconfiguration
5	Cookie security checks	Secure, HttpOnly, SameSite flags	A05 / A07
6	TLS / certificate analysis	Expiry, hostname match, weak protocol versions	A02: Cryptographic Failures
7	Passive technology detection	Confidence-rated framework/server fingerprinting	Information gathering
8	CORS misconfiguration check	Detects overly permissive cross-origin policies	A05: Security Misconfiguration
9	Common exposed-path detection	Probes .env, .git/config, backups (soft-404 baseline)	A05 / A01

5.3 Risk Scoring Methodology

WebSentinel computes its own transparent Risk Score on a 0–100 scale (higher = more secure). It is explicitly not CVSS. The score starts at 100 and subtracts a weighted penalty per finding; after four findings of the same severity, further findings of that severity count at half weight so one repeated misconfiguration does not distort the score. The result maps to a documented risk band:

Severity weight	Points	Risk bands (score → level)
Critical	25	90–100 → EXCELLENT
High	15	75–89 → GOOD
Medium	7	50–74 → MODERATE
Low	3	25–49 → POOR
Informational	0	0–24 → CRITICAL

Worked example (the demonstrated scan): two Medium findings (2 × 7 = 14) and five Low findings (four at full weight = 12, the fifth at half = 1.5) give a penalty of 27.5, so the score is 100 − 27.5 ≈ 72, mapped to MODERATE — exactly the value the tool reported.

5.4 Active vs. Passive Classification

WebSentinel is an active + non-intrusive scanner. The Nmap, HTTP and TLS modules send real traffic to the target (active), while header, cookie and technology detection only analyse responses already received (passive analysis). It deliberately avoids all intrusive techniques — no SQL-injection or XSS payloads, no brute-forcing, no fuzzing and no flooding — which keeps it safe against authorised targets while still requiring proper authorisation.

5.5 Scan Modes

WebSentinel offers three scan depths so the operator can trade speed against thoroughness. Quick is fast and light; Standard is the sensible default; Full adds the slower, deeper checks such as exposed-path detection.

Mode	Modules run	Typical use
Quick	HTTP, security headers, cookies, technology detection	A fast configuration sanity check
Standard (default)	Quick set + Nmap port/service scan + TLS/certificate analysis	Everyday assessment of a target
Full	Standard set + CORS check + curated common exposed-path detection	A deeper review before a report/presentation

The demonstrated scan in Section 7 used Full mode, which is why it also attempted the Nmap and exposed-path stages.

6. Impact Analysis

The weakness classes WebSentinel detects are individually low-to-medium severity, but in combination they meaningfully widen a site's attack surface. Most map to OWASP A05:2021 – Security Misconfiguration, the category that most consistently affects real-world web applications.

Weakness class	Security impact if left unaddressed
Missing Content-Security-	Removes a key defence-in-depth control against cross-site

Weakness class	Security impact if left unaddressed
Policy	scripting (XSS) and content injection.
Missing HSTS	Allows protocol-downgrade and SSL-stripping man-in-the-middle attacks on the first/HTTP connection.
Missing X-Frame-Options	Enables clickjacking — the site can be framed and overlaid by a malicious page.
Missing X-Content-Type-Options	Permits MIME-sniffing, which can turn served content into an XSS vector.
Missing Referrer/Permissions-Policy	Leaks referrer information and leaves powerful browser features unrestricted.
Cookie missing Secure/HttpOnly	Session cookies can be sent over HTTP or stolen via XSS, enabling session hijacking.
Weak cookie SameSite	Increases exposure to cross-site request forgery (CSRF).
Server version disclosure	Gives attackers precise fingerprints to look up version-specific exploits.
Weak/expired TLS	Undermines confidentiality and integrity; expired certificates enable MITM.
Exposed sensitive paths	Direct leakage of credentials, source code or configuration (e.g. .env, .git).

6.1 Business & Operational Impact

- Reputational and trust damage if customer sessions or data are compromised through avoidable misconfiguration.
- Regulatory and compliance exposure — many of these controls are expected by standards such as PCI-DSS and are checked in audits.
- Increased incident likelihood — each disclosed banner or missing header lowers the effort required for an attacker to progress.
- The cumulative effect matters most — a single missing header is minor, but several together indicate a systemic hardening gap.

6.2 OWASP Top 10 Context

Almost every WebSentinel finding maps to A05:2021 – Security Misconfiguration, which the OWASP Top 10 lists among the most prevalent risk categories in modern web applications. Misconfiguration is attractive to attackers precisely because it requires no sophisticated exploit — a missing header, a verbose banner or an exposed file is simply information or a weakness handed over for free. By focusing on this category, WebSentinel targets the issues most likely to be present on a real site and most cheaply fixed, which is why it is a sensible first-stage assessment tool even though it does not attempt exploitation.

7. Proof of Concept (PoC)

The following is a real WebSentinel run performed on Kali Linux against scanme.nmap.org (an authorised target). The screenshots follow the actual workflow, from launching the tool to the generated results.

7.1 Scan Metadata

Field	Value
Target	http://scanme.nmap.org
Resolved IP	45.33.32.156
Scan mode	Full
Scanned at	2026-09-16 07:24:12 UTC
Duration	131.1 seconds
Security score	**72 / 100**
Risk level	**MODERATE**
Report generated	reports/scan_98312798.pdf

7.2 Workflow Screenshots

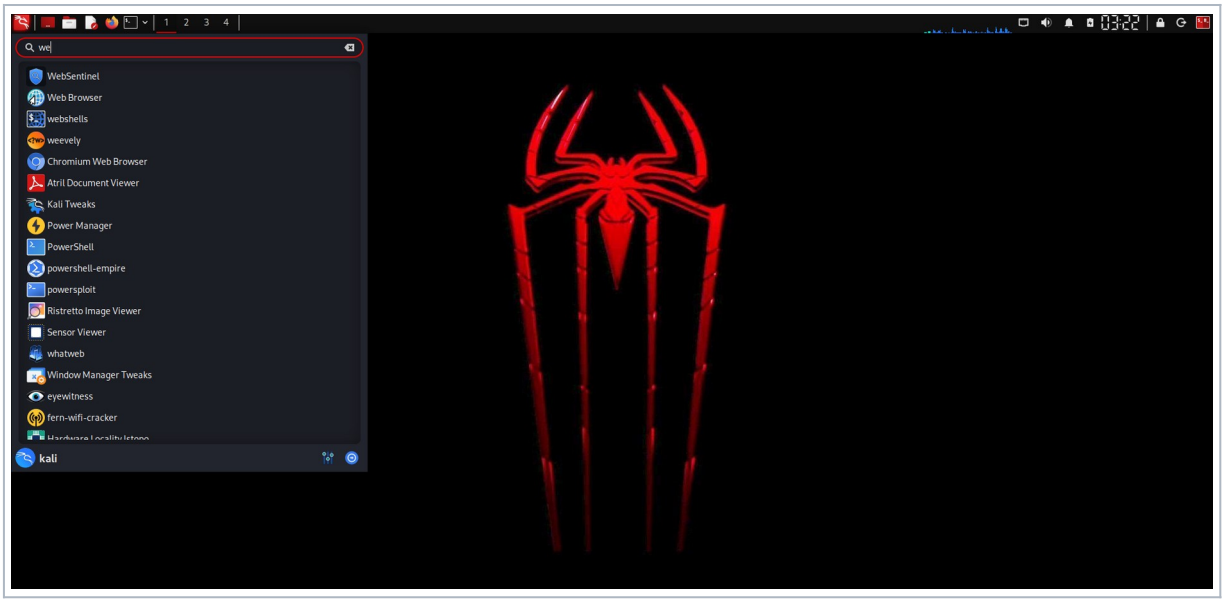


Figure 1 — WebSentinel installed as a Kali application: it appears in the menu (searching "we") alongside standard tools, launchable like nmap or Burp Suite.

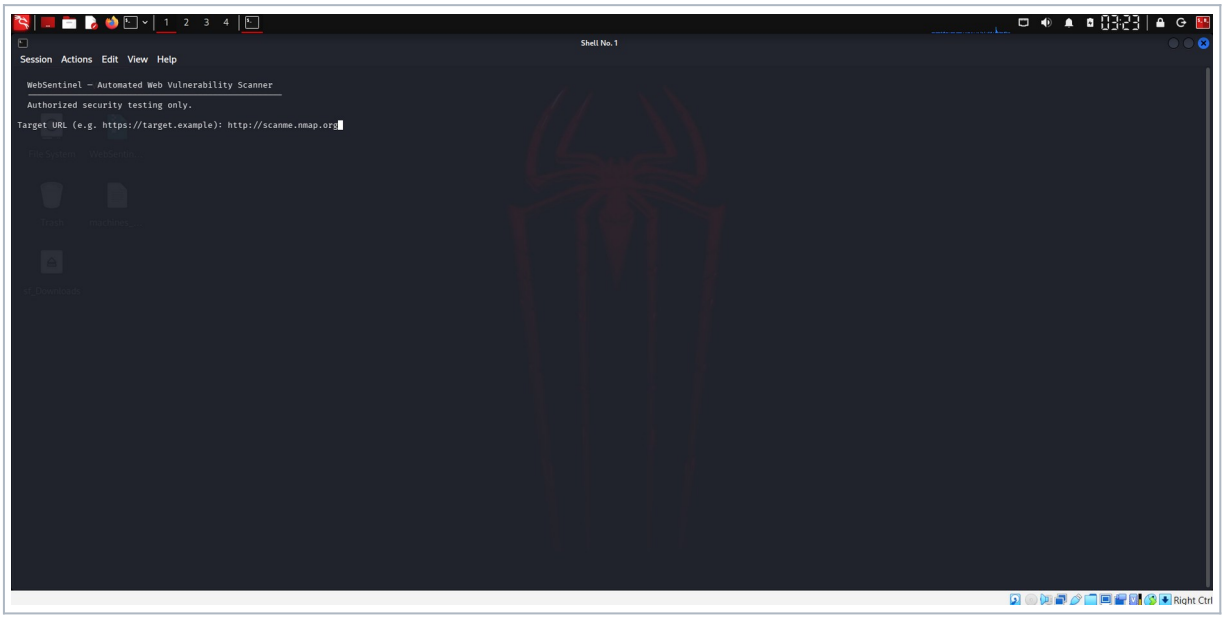


Figure 2 — Launching WebSentinel: the banner, the authorised-use notice, and the target URL prompt (http://scanme.nmap.org).

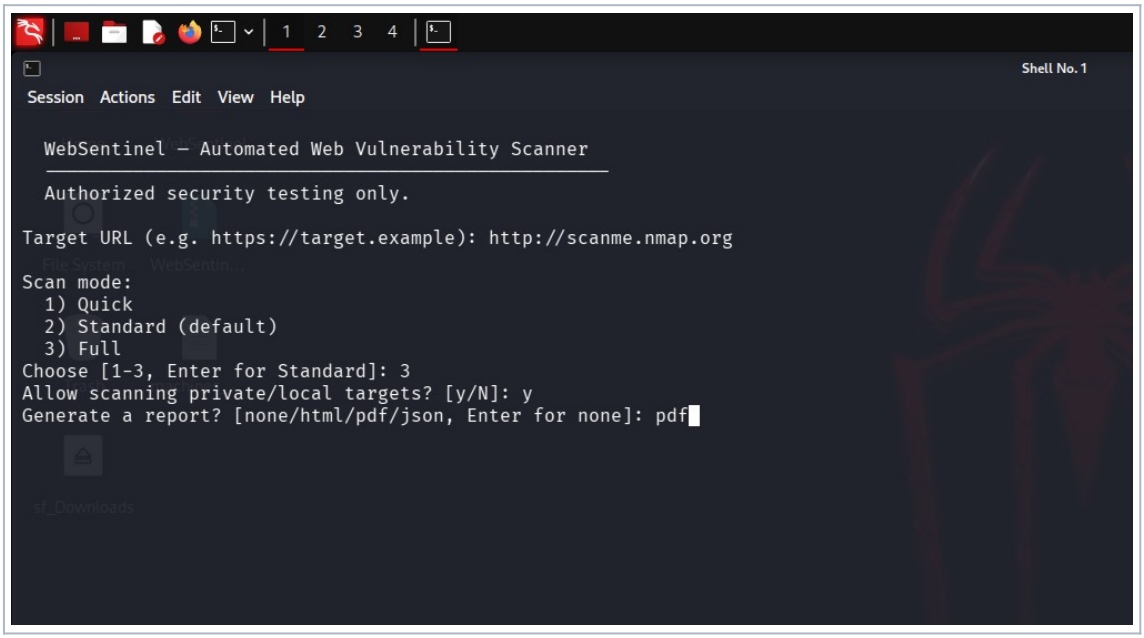


Figure 3 — Interactive scan configuration: scan mode set to Full (option 3) and PDF report selected.

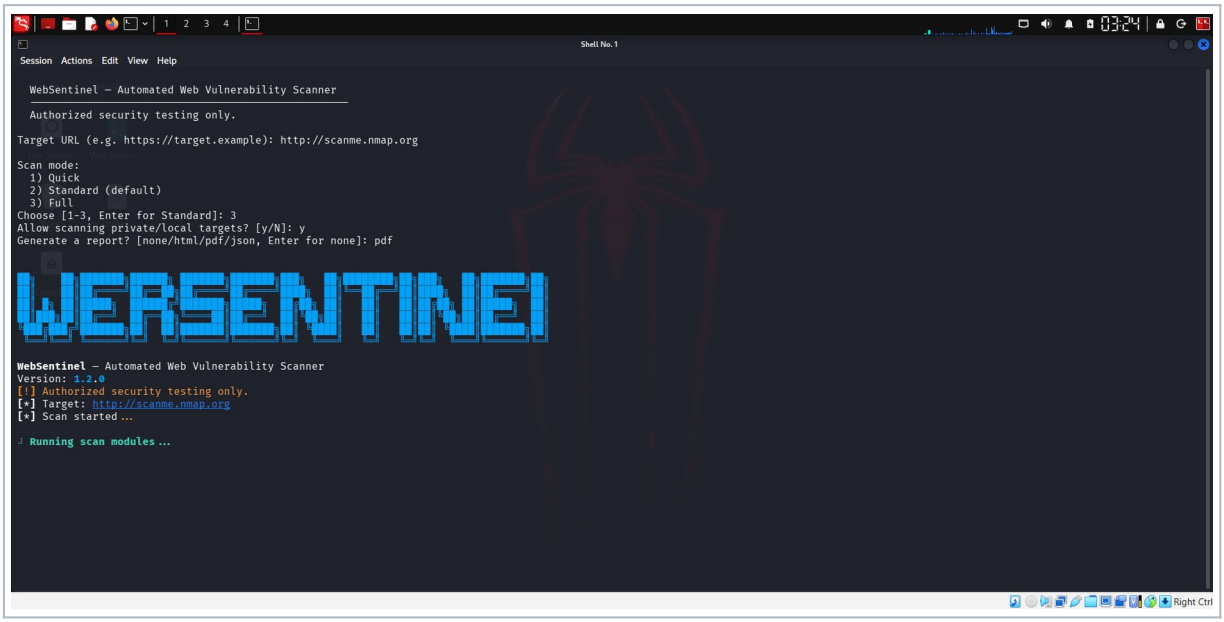


Figure 4 — Scan starting: WebSentinel v1.2.0 confirms the target and begins running its scan modules.

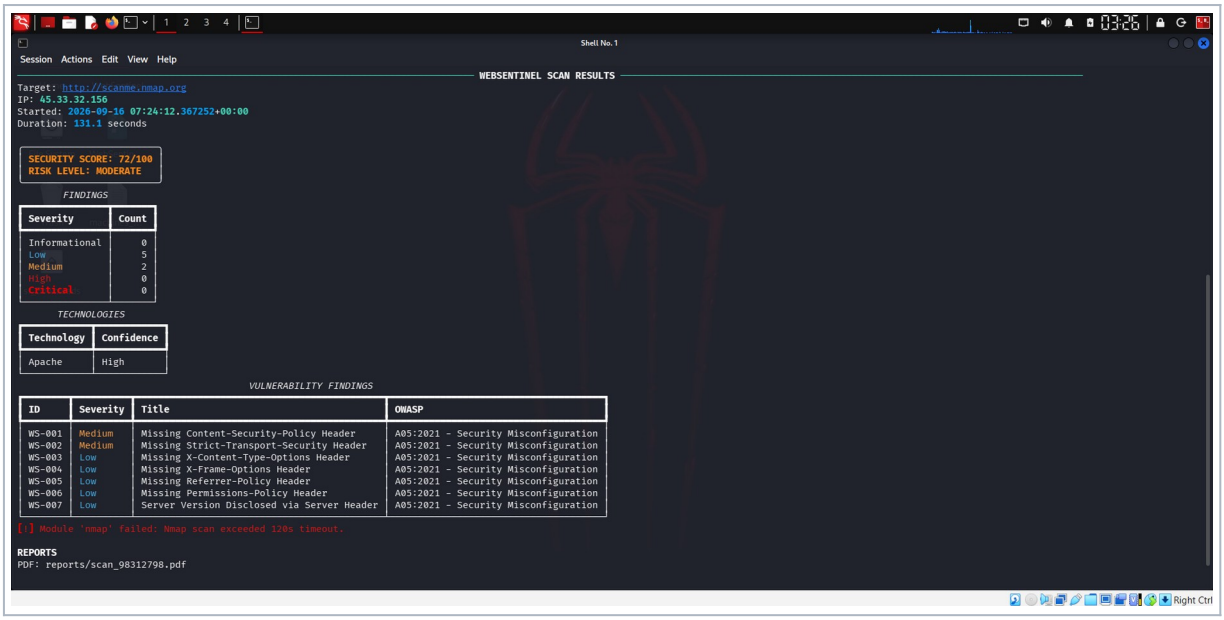


Figure 5 — Results: security score 72/100 (MODERATE), findings summary, detected technology (Apache, High confidence) and the seven OWASP-mapped findings, with the PDF report path.

7.3 Vulnerability Findings

The scan produced seven findings, all mapped to OWASP A05:2021 – Security Misconfiguration:

ID	Severity	Finding	OWASP
WS-001	**Medium**	Missing Content-Security-Policy header	A05:2021
WS-002	**Medium**	Missing Strict-Transport-Security header	A05:2021
WS-003	**Low**	Missing X-Content-Type-Options header	A05:2021
WS-004	**Low**	Missing X-Frame-Options header	A05:2021

ID	Severity	Finding	OWASP
WS-005	Low	Missing Referrer-Policy header	A05:2021
WS-006	Low	Missing Permissions-Policy header	A05:2021
WS-007	Low	Server version disclosed via Server header	A05:2021

These reflect the target's live configuration: several recommended security headers are absent and the exact Apache version is disclosed in the Server banner. None are critical, which is why the host scores a MODERATE 72/100.

7.4 Detailed Findings

Each finding below is reproduced from the generated report, showing the standardised fields WebSentinel records — severity, confidence, OWASP mapping, description, evidence, impact and remediation.

WS-001 — Missing Content-Security-Policy Header

Field	Detail
Severity	Medium
Confidence	High
OWASP	A05:2021 – Security Misconfiguration
Description	The response does not include a 'Content-Security-Policy' header, which helps mitigate XSS and data-injection attacks.
Evidence	Header 'Content-Security-Policy' absent from HTTP response.
Impact	Increases exposure to the class of attacks this header is designed to mitigate.
Remediation	Configure the web server or application to send a 'Content-Security-Policy' header with an appropriate policy.

WS-002 — Missing Strict-Transport-Security Header

Field	Detail
Severity	Medium
Confidence	High
OWASP	A05:2021 – Security Misconfiguration
Description	The response does not include a 'Strict-Transport-Security' header, which ensures browsers only connect over HTTPS.
Evidence	Header 'Strict-Transport-Security' absent from HTTP response.
Impact	Increases exposure to the class of attacks this header is designed to mitigate.
Remediation	Configure the web server or application to send a 'Strict-

Field	Detail
	Transport-Security' header with an appropriate policy.

WS-003 — Missing X-Content-Type-Options Header

Field	Detail
Severity	**Low**
Confidence	High
OWASP	A05:2021 – Security Misconfiguration
Description	The response does not include an 'X-Content-Type-Options' header, which prevents MIME-type sniffing.
Evidence	Header 'X-Content-Type-Options' absent from HTTP response.
Impact	Increases exposure to the class of attacks this header is designed to mitigate.
Remediation	Configure the web server or application to send an 'X-Content-Type-Options: nosniff' header.

WS-004 — Missing X-Frame-Options Header

Field	Detail
Severity	**Low**
Confidence	High
OWASP	A05:2021 – Security Misconfiguration
Description	The response does not include an 'X-Frame-Options' header, which helps mitigate clickjacking (partially superseded by CSP frame-ancestors).
Evidence	Header 'X-Frame-Options' absent from HTTP response.
Impact	Increases exposure to the class of attacks this header is designed to mitigate.
Remediation	Send an 'X-Frame-Options' header (or a CSP 'frame-ancestors' directive).

WS-005 — Missing Referrer-Policy Header

Field	Detail
Severity	**Low**
Confidence	High
OWASP	A05:2021 – Security Misconfiguration
Description	The response does not include a 'Referrer-Policy' header, which

Field	Detail
	controls how much referrer data leaks to other sites.
Evidence	Header 'Referrer-Policy' absent from HTTP response.
Impact	Increases exposure to the class of attacks this header is designed to mitigate.
Remediation	Send a 'Referrer-Policy' header with an appropriate value (e.g. strict-origin-when-cross-origin).

WS-006 — Missing Permissions-Policy Header

Field	Detail
Severity	**Low**
Confidence	High
OWASP	A05:2021 – Security Misconfiguration
Description	The response does not include a 'Permissions-Policy' header, which restricts access to sensitive browser features/APIs.
Evidence	Header 'Permissions-Policy' absent from HTTP response.
Impact	Increases exposure to the class of attacks this header is designed to mitigate.
Remediation	Send a 'Permissions-Policy' header restricting features the site does not use.

WS-007 — Server Version Disclosed via Server Header

Field	Detail
Severity	**Low**
Confidence	Medium
OWASP	A05:2021 – Security Misconfiguration
Description	The 'Server' header discloses specific software/version information: 'Apache/2.4.7 (Ubuntu)'.
Evidence	Server: Apache/2.4.7 (Ubuntu)
Impact	Helps an attacker fingerprint the stack and target known vulnerabilities for that version.
Remediation	Configure the server to omit or generalise the 'Server' header (e.g. ServerTokens Prod).

8. Recommendations — Security Measures & Mitigation

Based on the findings above, the following mitigations are recommended. Most are low-effort, high-value configuration changes at the web-server or application layer.

8.1 Fixing the Detected Weakness Classes

Weakness	Recommended mitigation
Missing security headers	Add CSP, HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy and Permissions-Policy at the server/CDN or via a security-headers middleware.
Insecure cookies	Set Secure, HttpOnly and an appropriate SameSite (Lax/Strict) attribute on all session cookies.
Server version disclosure	Suppress or generalise the Server banner (e.g. ServerTokens Prod on Apache).
TLS configuration	Enforce HTTPS, enable HSTS, keep certificates valid, and disable weak/legacy protocols (SSLv3, TLS 1.0/1.1) and ciphers.
Exposed sensitive files	Remove or block access to .env, .git/, backups and admin/debug endpoints; never deploy source-control or config files to web roots.
Unnecessary open ports	Close ports that are not required and keep exposed services (SSH, web server) patched to current versions.

8.2 General Hardening & Process Recommendations

- Apply security headers and cookie flags as framework defaults, not per-page.
- Adopt a regular scanning cadence — integrate a scanner such as WebSentinel into development and pre-release checks.
- Keep all components patched: web server, runtime, libraries and OS.
- Follow the principle of least privilege and minimise the exposed attack surface.
- Use a Web Application Firewall (WAF) as defence-in-depth, not a substitute for fixing root causes.
- Always obtain explicit written authorisation before scanning any system you do not own — active scanning is legally sensitive even when non-destructive.

9. Code & Tools Used

WebSentinel is written entirely in Python 3 and uses well-established open-source libraries. It is packaged for real use — installable globally so that websentinel runs from any terminal, with an application-menu launcher and icon on Kali Linux.

9.1 Technology Stack & Libraries

Component	Library / Tool	Purpose
Language / runtime	Python 3.11+	Core implementation
HTTP client	requests	Sending HTTP/HTTPS requests
CLI framework	Typer	Command-line interface and flags
Terminal UI	Rich	Formatted tables, colour, banner output
Port scanning	Nmap (subprocess)	Port/service enumeration
HTML reports	Jinja2	Templated HTML report generation
PDF reports	ReportLab	PDF report generation
Storage	SQLite	Local scan-history database
Testing	pytest	Automated unit & integration tests
Packaging / CI	pipx, Docker, GitHub Actions	Global install, containerisation, CI

9.2 Key Code — Target Validation (SSRF Protection)

Every target must pass validation before any scanner touches it. Private, loopback, link-local and cloud-metadata addresses are refused by default:

```
# core/target.py
BLOCKED_IPS = {
    "169.254.169.254",   # AWS / Azure / GCP metadata
    "100.100.100.200",   # Alibaba Cloud metadata
}

def _is_disallowed_ip(ip_obj) -> bool:
    return (ip_obj.is_private or ip_obj.is_loopback
            or ip_obj.is_link_local or ip_obj.is_unspecified
            or ip_obj.is_multicast or ip_obj.is_reserved
            or str(ip_obj) in BLOCKED_IPS)

def validate_target(raw_target, allow_private=False):
    url = normalize_url(raw_target)
    parsed = urlparse(url)
    if parsed.scheme not in ALLOWED_SCHEMES:
        raise TargetValidationError("Only http/https are allowed.")
    # ... resolve host, refuse disallowed IPs unless allow_private ...
```

9.3 Key Code — WebSentinel Risk Score

```
# core/scoring.py
```

```
SEVERITY_WEIGHTS = {CRITICAL: 25, HIGH: 15, MEDIUM: 7, LOW: 3, INFO: 0}
RISK_BANDS = [(90,100,"EXCELLENT"), (75,89,"GOOD"),
              (50,74,"MODERATE"), (25,49,"POOR"), (0,24,"CRITICAL")]

def calculate_score(findings):
    penalty = 0.0
    for severity, count in by_severity.items():
        weight = SEVERITY_WEIGHTS[severity]
        full = min(count, 4)          # full weight for first 4
        half = max(0, count - 4)      # half weight thereafter
        penalty += full*weight + half*weight*0.5
    score = max(0, round(100 - penalty))
    return score, risk_level_for_score(score)
```

9.4 Key Code — Turning Evidence into a Finding

The checks layer converts a raw scanner observation into a standardised Finding with severity, OWASP mapping and remediation:

```
# checks/headers.py
def build_findings(header_results):
    findings = []
    for hr in header_results:
        if hr.present:
            continue
        findings.append(Finding(
            title=f"Missing {hr.header} Header",
            severity=_SEVERITY_MAP.get(hr.severity_if_missing, LOW),
            confidence=Confidence.HIGH,
            owasp=owasp.A05,
            evidence=f"Header '{hr.header}' absent from HTTP response.",
            remediation=f"Send a '{hr.header}' header with a suitable policy."))
    return findings
```

9.5 Key Code — Safe Nmap Invocation

Nmap is called through an argument array (never shell=True), with conservative flags and a hard timeout — no stealth, evasion or exploit scripts:

```
# scanners/nmap.py  —  conservative, non-destructive defaults
args = [self.binary, "-Pn", "-T3", "--top-ports", str(top_ports)]
if service_detection:
    args.append("-sV")            # version detection (safe)
args += ["-oX", "-", host]        # XML to stdout for reliable parsing

proc = subprocess.run(
    args, capture_output=True, text=True,
    timeout=self.timeout, check=False)   # array args, never shell=True
```

9.6 Installation & Usage Commands

```
# Install (editable, with PDF support)
pip install -e ".[pdf]"

# Global install so 'websentinel' works from any terminal
```

```
pipx install ".[pdf]"
bash packaging/install_desktop_entry.sh   # menu entry + icon

# Run a full scan and produce a PDF report
websentinel scan -u http://scanme.nmap.org --full --report pdf
websentinel --version                      # 1.2.0
```

9.7 Project Structure

The codebase is organised so that data collection (scanners), interpretation (checks) and output (reports) are cleanly separated. The package layout is:

```
WebSentinel/
├── src/websentinel/
│   ├── cli.py            # Typer CLI + interactive prompts
│   ├── engine.py         # orchestrates the scan pipeline
│   ├── config.py         # loads config/config.yaml
│   ├── core/
│   │   ├── target.py     # target validation + SSRF guard
│   │   ├── scoring.py    # WebSentinel Risk Score
│   │   ├── models.py     # Finding / Severity data models
│   │   └── logging.py
│   ├── scanners/         # raw data collection
│   │   ├── nmap.py  http.py  headers.py  cookies.py
│   │   └── tls.py   technology.py  common_paths.py
│   ├── checks/           # observations -> standardized Findings
│   │   ├── headers.py  cookies.py  tls.py
│   │   └── disclosure.py  common_paths.py  owasp.py
│   ├── database/sqlite.py      # scan history
│   └── reports/                # json / html / pdf report writers
├── tests/                      # pytest suite (unit + integration)
├── packaging/                  # global install, .desktop entry, icons
├── docs/                       # architecture / methodology / security
├── config/config.yaml          # scan configuration
├── Dockerfile   .github/workflows/ci.yml
└── pyproject.toml  README.md  LICENSE
```

This separation is what makes the tool maintainable and easy to explain: a scanner only gathers evidence, a matching check turns that evidence into a Finding with its OWASP mapping and remediation, and the engine simply runs them in sequence and hands the results to the scorer and report writers.

10. Comparison with Existing Kali Tools

Kali Linux ships mature scanners such as Nikto, OWASP ZAP and Burp Suite. WebSentinel does not aim to replace them; it is a smaller, transparent, from-scratch tool that a learner can fully understand. The honest positioning:

Tool	Strength	How WebSentinel differs
Nikto	Thousands of known-path signatures	Probes a small curated path set the developer wrote and understands, plus OWASP

Tool	Strength	How WebSentinel differs
		mapping & scoring
OWASP ZAP	Full proxy, active/passive, spidering	CLI-only, lightweight, non-intrusive, no exploitation
Burp Suite	Interactive testing & interception	Automated, single-command, report-focused

The distinctive value of WebSentinel is clarity, not raw coverage: it consolidates port, header, cookie, TLS, technology and exposed-path checks into one command, ranks findings by severity, maps them to the OWASP Top 10 and produces a reproducible score — in code the author wrote and can explain end to end.

11. Limitations, Testing & Validation

11.1 Testing & Quality Assurance

- 40 automated tests (unit + integration) across scoring, target validation, headers, cookies, TLS, CORS, exposed-paths, reports, rate-limiting and packaging — all passing.
- Security-specific tests: cloud-metadata IP (169.254.169.254) is hard-blocked even in lab mode; HTML report output is escaped to prevent report injection.
- Manual end-to-end verification against scanme.nmap.org and other authorised test targets, with real generated PDF/HTML/JSON reports.

11.2 Known Limitations

- Automated, non-destructive checks only — no authentication, authorization, business-logic testing or exploitation.
- Results may contain false positives/negatives and should be reviewed by a qualified analyst.
- The Nmap module is time-bounded (120 s); against slower hosts it can time out, as seen in the final run — the scan still completes and reports its other findings.

11.3 Real Bugs Found & Fixed (v1.0.0 → v1.2.0)

#	Bug	Fix
1	--version flag misrouted by the CLI argument shim	Added it to the command exceptions so it no longer triggers a scan
2	ASCII banner corrupted on narrow terminals	Width check with a plain-text fallback
3	--only headers,cookies produced zero findings	Auto-keep the http module alive as an implicit dependency
4	HTML report crashed under a real pipx install	Bundled the Jinja2 template inside the Python package
5	Rate limiting configured but never enforced	Implemented and verified enforcement

#	Bug	Fix
		with timing tests
6	Version number not bumped between builds	Adopted real version bumps (now 1.2.0)

11.4 Deliverables Checklist

The report is structured directly around the deliverables in the BIA project brief. The table below maps each required deliverable to where it is addressed.

BIA deliverable	Status	Where addressed
Abstract	**Complete**	Section 1
Research (site details)	**Complete**	Section 3
Data Collection (tech stack)	**Complete**	Section 4
Impact Analysis	**Complete**	Section 6
Recommendation / Mitigation	**Complete**	Section 8
Code / Tools used	**Complete**	Section 9
Proof of Concept (screenshots)	**Complete**	Section 7 + Appendix A
References	**Complete**	Section 13
Presentation (report format)	**Complete**	Whole document

12. Conclusion

WebSentinel meets its objective: a lightweight, easy-to-use Python tool that performs a genuine website vulnerability scan and reports common security weaknesses with OWASP mapping, a documented risk score and multi-format reporting. It was validated against an authorised target, packaged for real day-to-day use on Kali Linux, and hardened with safety guardrails so it stays firmly on the defensive, non-intrusive side of security testing.

The project also produced two lasting lessons. Technically, small misconfigurations compound — a set of missing headers and a disclosed version banner together tell an attacker a great deal. Ethically and legally, active scanning — even when non-destructive — requires explicit authorisation; the demonstration used scanme.nmap.org precisely because it is a host the Nmap Project permits people to scan. That boundary is a core professional principle, not a formality.

12.1 Future Work

- Add authenticated scanning (with supplied credentials) to assess logged-in areas of an application.

- Expand the exposed-path list and add optional integration with public vulnerability databases for version-based checks.
- Provide a lightweight web dashboard for browsing scan history stored in SQLite.
- Add scheduling so recurring scans of an owned target can be automated and their scores trended over time.

These extensions would deepen the tool without changing its guiding principle: transparent, non-intrusive, authorised assessment that a user can fully understand.

13. References

1. Nmap — scanme.nmap.org / official documentation — https://nmap.org/book/toc.html
2. OWASP Top 10 (2021) — https://owasp.org/Top10/
3. OWASP A05:2021 – Security Misconfiguration — https://owasp.org/Top10/A05_2021-Security_Misconfiguration/
4. OWASP Secure Headers Project — https://owasp.org/www-project-secure-headers/
5. MDN — Content-Security-Policy — https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Content-Security-Policy
6. MDN — Strict-Transport-Security (HSTS) — https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security
7. MDN — Set-Cookie (Secure, HttpOnly, SameSite) — https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Set-Cookie
8. Nikto web scanner — https://github.com/sullo/nikto
9. OWASP ZAP — https://www.zaproxy.org/
10. Python requests — https://requests.readthedocs.io/
11. Typer CLI framework — https://typer.tiangolo.com/
12. Rich terminal library — https://rich.readthedocs.io/
13. Jinja2 templating — https://jinja.palletsprojects.com/
14. ReportLab PDF toolkit — https://www.reportlab.com/

Appendix A — Full Scan Report Output

This appendix reproduces the WebSentinel security assessment report generated for the demonstrated scan (reports/scan_98312798.pdf), for reference and verification.

A.1 Report Header

Authorization Disclaimer: This assessment must only be performed against systems the requester owns or has explicit written authorisation to test. Results should be validated by a qualified analyst before acting on them.

A.2 Executive Summary

Field	Value
Target	http://scanme.nmap.org
Resolved IP	45.33.32.156
Scan Mode	full
Scanned At	2026-09-16 07:24:12 UTC
Duration	131.1s
Security Score	**72/100**
Risk Level	**MODERATE**

A.3 Finding Summary

Severity	Count
Informational	0
Low	5
Medium	2
High	0
Critical	0

A.4 Vulnerability Findings (verbatim)

```
[Medium] Missing Content-Security-Policy Header (WS-001)
OWASP: A05:2021 - Security Misconfiguration    Confidence: High
Description: The response does not include a 'Content-Security-Policy'
  header. Helps mitigate XSS and data-injection attacks.
Evidence: Header 'Content-Security-Policy' absent from HTTP response.
Remediation: Send a 'Content-Security-Policy' header with a suitable policy.

[Medium] Missing Strict-Transport-Security Header (WS-002)
OWASP: A05:2021 - Security Misconfiguration    Confidence: High
Description: The response does not include a 'Strict-Transport-Security'
  header. Ensures browsers only connect over HTTPS.
```

```
Evidence: Header 'Strict-Transport-Security' absent from HTTP response.

[Low] Missing X-Content-Type-Options Header (WS-003)
OWASP: A05:2021 - Security Misconfiguration    Confidence: High
Description: Prevents MIME-type sniffing.
Evidence: Header 'X-Content-Type-Options' absent from HTTP response.

[Low] Missing X-Frame-Options Header (WS-004)
OWASP: A05:2021 - Security Misconfiguration    Confidence: High
Description: Helps mitigate clickjacking (superseded by CSP frame-ancestors).
Evidence: Header 'X-Frame-Options' absent from HTTP response.

[Low] Missing Referrer-Policy Header (WS-005)
OWASP: A05:2021 - Security Misconfiguration    Confidence: High
Description: Controls how much referrer data leaks to other sites.
Evidence: Header 'Referrer-Policy' absent from HTTP response.

[Low] Missing Permissions-Policy Header (WS-006)
OWASP: A05:2021 - Security Misconfiguration    Confidence: High
Description: Restricts access to sensitive browser features/APIs.
Evidence: Header 'Permissions-Policy' absent from HTTP response.

[Low] Server Version Disclosed via Server Header (WS-007)
OWASP: A05:2021 - Security Misconfiguration    Confidence: Medium
Description: The 'Server' header discloses 'Apache/2.4.7 (Ubuntu)'.
Evidence: Server: Apache/2.4.7 (Ubuntu)
Remediation: Configure the server to omit or generalise the 'Server' header.
```

A.5 Limitations (verbatim)

WebSentinel performs automated, non-destructive checks only. It does not perform authentication testing, business-logic testing, or exploitation. Results may contain false positives/negatives and should be reviewed by a qualified security professional.

Appendix B — Environment & Reproduction

The demonstrated assessment can be reproduced on any Kali Linux (or Debian-based) system with the environment below. The target must remain scanme.nmap.org or another host you own or are authorised to test.

B.1 Test Environment

Component	Value
Operating system	Kali Linux (rolling), running under VirtualBox
Python	3.11+
Tool version	WebSentinel 1.2.0
External dependency	nmap (installed via apt)
Install method	pipx (global command) + desktop menu entry
Target	http://scanme.nmap.org (authorised)

B.2 Reproduction Steps

```
# 1) Extract the project and enter it
cd ~/Desktop/WebSentinel

# 2) Create a virtual environment and install (with PDF support)
python3 -m venv venv
source venv/bin/activate
pip install -e ".[pdf]"

# 3) Install globally + add the Kali application-menu entry
pipx install ".[pdf]" --force
bash packaging/install_desktop_entry.sh

# 4) Confirm the version, then run the full scan used in this report
websentinel --version                # -> 1.2.0
websentinel scan -u http://scanme.nmap.org --full --report pdf

# 5) The PDF report is written under ./reports/
```

Running the command above reproduces the assessment in Section 7: WebSentinel resolves the host, runs its scan modules, scores the result, prints the findings table and writes a PDF report to the reports/ directory.